

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «КубГУ»)

Факультет компьютерных технологий и прикладной математики
Кафедра вычислительных технологий

ЛАБОРАТОРНАЯ РАБОТА №7
Дисциплина: «Методы поисковой оптимизации»

Работу выполнили: _____ Долгов М.И. и Костров А.А.

Направление подготовки: 02.03.02 Фундаментальная информатика и
информационные технологии

Преподаватель: _____ Нигодин Е.А.

Краснодар
2026

Тема. Канонический алгоритм бактериальной оптимизации

Цель. Реализовать приложение с графическим интерфейсом, демонстрирующее применение бактериального для решения задачи поиска минимума функции.

Алгоритм.

Пусть требуется найти минимум целевой функции:

$$f(\mathbf{x}) \rightarrow \min, \mathbf{x} \in D \subset \mathbb{R}^n$$

где:

- n — размерность пространства поиска;
- D — допустимая область, задаваемая ограничениями:
- $D = \{\mathbf{x} \mid x_{\min}^i \leq x_i \leq x_{\max}^i, i = 1, \dots, n\}$

Поведение бактерий обусловлено механизмом, который называется бактериальным хемотаксисом и представляет собой двигательную реакцию микроорганизмов на химический раздражитель.

Рассмотрим задачу многомерной глобальной безусловной оптимизации, полагая, что значения фитнес функции $f(X)$ должны быть минимизированны.

Канонический алгоритм бактериальной оптимизации BFO основан на использовании трёх следующих основных механизмов: хемотаксис, репродукция, ликвидация и рассеивание.

- Пусть $|S|$ - множество всех бактерий текущей популяции

Хемотаксис:

- Процедура хемотаксиса реализует в алгоритме BFO локальную оптимизацию. Следующие положения бактерии $X'_{i,r,l} = X_{i,r,l} + \lambda_i \frac{V_i}{||V_i||_E}$, где V_i – текущий направляющий вектор шага хемотаксиса бактерии. При плавании бактерии, вектор V_i остаётся неизменным, плавание продолжается до тех пор, пока происходит уменьшение значений фитнес-функции.
- Величина хемотаксиса изменяется в процессе поиска по закону с ростом числа итераций t
- Здесь:
 - i – номер бактерии в колонии
 - r – шаг репродукции
 - l – шаг ликвидации и рассеивания

Репродукция:

- Назовём текущим состоянием здоровья h_i бактерии s_i .
- В ходе репродукции, на $r+1$ шаге репродукции вторая половина бактерий популяции погибает, а каждая бактерия первой половины расщепляется на две одинаковых бактерии с одинаковыми координатами, равными координатам расщеплённого агента
- В ходе выполнения репродукции, общее число бактерий в популяции остаётся неизменным и равным $|S|$

Ликвидация:

- Механизм ликвидации и рассеивания включается после выполнения определённого числа процедур репродукции
- С заданной вероятностью ξ случайным образом выбираем $p < |S|$ бактерий и уничтожаем их. Вместо каждой из уничтоженных бактерий в случайно выбранной точке пространства поиска создаём новую.

Структура алгоритма:

- 1) Инициализировать популяцию с помощью начальных положений бактерий $X_{i,0,0}(0), i[1: |S|]$ в исследуемом диапазоне
- 2) Выполнение t числа шагов хемотаксиса каждой бактерии
- 3) Выполнение t^r числа шагов репродукции
- 4) Выполнение t^l числа шагов ликвидации и рассеивания.

Основные параметры алгоритма

$t_chemotaxis$ # общее число шагов хемотаксиса ($\hat{t}$)
 $t_reproduction$ # общее число шагов репродукции ($\hat{t}^r$)
 $t_elimination$ # общее число шагов ликвидации и рассеивания ($\hat{t}^l$)
 λ_i # величины шагов хемотаксиса, $i \in [1, S]$
 $p_eliminate$ # вероятность ликвидации (ξ_e)
 $n_eliminate$ # число бактерий, уничтожаемых при ликвидации (n)
 x_min # нижняя граница координат (x_k^{-})
 x_max # верхняя граница координат (x_k^{+})

1. Программа представляет собой desktop-приложение, реализованное на языке Python с использованием фреймворка PySide6 и библиотеки pyqtgraph для визуализации. Приложение предназначено для демонстрации алгоритмов оптимизации функций в 3D-пространстве (рисунок 1).

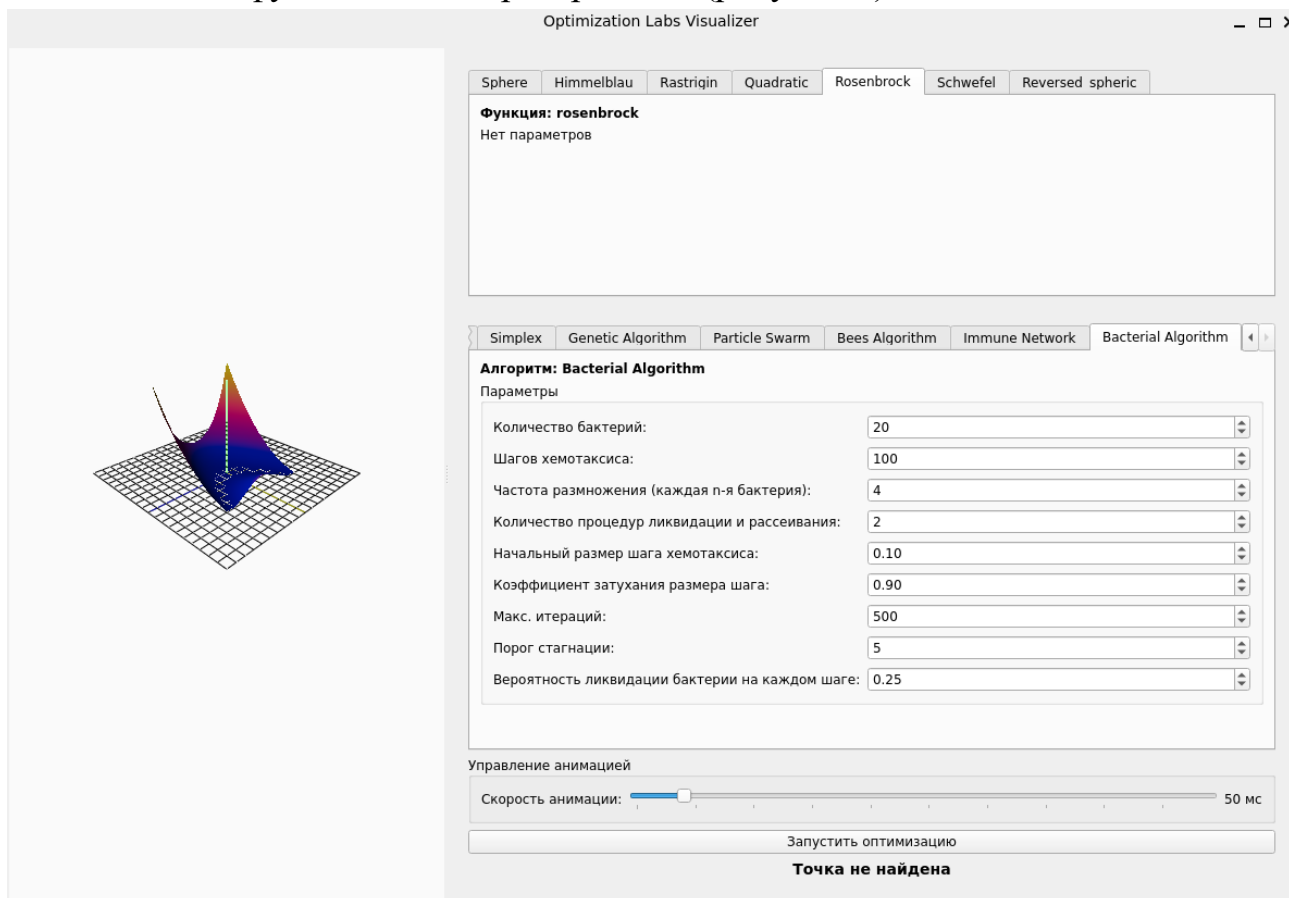


Рисунок 1 — Главное окно приложения

2. Доступные функции: Sphere, Himmelblau, Rastrigin, Quadratic и Schwefel, Reversed spheric (рисунок 2).

Sphere	Himmelblau	Rastrigin	Quadratic	Rosenbrock	Schwefel	Reversed spheric
--------	------------	-----------	-----------	------------	----------	------------------

Функция: rosenbrock
Нет параметров

Рисунок 2 — Доступные функции

3. Доступные алгоритмы оптимизации: градиентный спуск с постоянным шагом, симплекс-метод, генетический алгоритм, алгоритм роя частиц, пчелиный алгоритм, бактериальный алгоритм (рисунок 3).

Simplex	Genetic Algorithm	Particle Swarm	Bees Algorithm	Immune Network	Bacterial Algorithm
---------	-------------------	----------------	----------------	----------------	---------------------

Алгоритм: Bacterial Algorithm
Параметры

Количество бактерий:	20
Шагов хемотаксиса:	100
Частота размножения (каждая n-я бактерия):	4
Количество процедур ликвидации и рассеивания:	2
Начальный размер шага хемотаксиса:	0.10
Коэффициент затухания размера шага:	0.90
Макс. итераций:	500
Порог стагнации:	5
Вероятность ликвидации бактерии на каждом шаге:	0.25

Рисунок 3 — Доступные алгоритмы оптимизации

4. Запустим алгоритм поиска минимума функции Розенброка (рисунок 4).

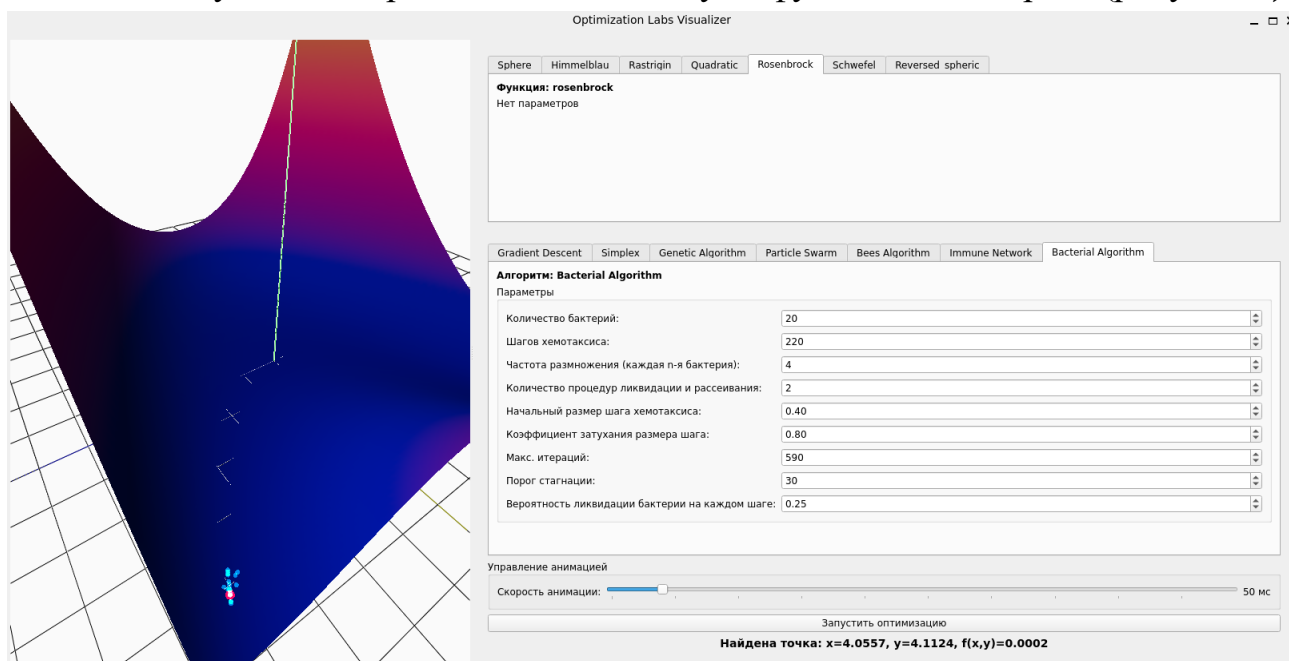


Рисунок 4 — Запуск алгоритма оптимизации

Вывод. В ходе данной лабораторной работы было реализовано приложение с графическим интерфейсом, демонстрирующее применение бактериального алгоритма для решения задачи поиска минимума функции.

Листинг программы.

```
import numpy as np
from typing import Generator, List

class BacterialOptimization:
    """Алгоритм бактериальной оптимизации (Bacterial Foraging Optimization)"""
    def __init__(
        self,
        func,
        func_params,
        bounds: list[tuple],
        n_bacteria: int = 20,
        n_chemotaxis: int = 100,
        n_reproduction: int = 4,
        n_elimination: int = 2,
        step_size: float = 0.1,
        decay_rate: float = 0.9,
        max_iter: int = 500,
        stagnation: int = 20,
        elim_prob: float = 0.25,
    ):
        self.func = func
        self.func_params = func_params
        self.bounds = np.array(bounds, dtype=float)
        self.dim = len(bounds)
        self.n_bacteria = n_bacteria
        self.n_chemotaxis = n_chemotaxis
        self.n_reproduction = n_reproduction
        self.n_elimination = n_elimination
        self.step_size = step_size
        self.decay_rate = decay_rate
        self.max_iter = max_iter
        self.stagnation = stagnation
        self.elim_prob = elim_prob
        self.stagnation_param = 3

    def _random_point(self) -> np.ndarray:
        lo, hi = self.bounds[:, 0], self.bounds[:, 1]
        return lo + np.random.rand(self.dim) * (hi - lo)

    def _fitness(self, p: np.ndarray):
        return -self.func(*p, **self.func_params)

    def _chemotaxis_step(self, bacteria_pos: np.ndarray, step_size: float,
direction: np.ndarray) -> np.ndarray:
```

```

"""
Хемотаксис: движение бактерии в заданном направлении с заданным размером
шага
"""
direction = direction / (np.linalg.norm(direction) + 1e-10)

# Новое положение с учётом размера шага
new_pos = bacteria_pos + step_size * direction
new_pos = np.clip(new_pos, self.bounds[:, 0], self.bounds[:, 1])
return new_pos

def _health_status(self, fitness_values: List[float], idx: int) -> float:
    """
    Состояние здоровья бактерии: сумма значений функции приспособленности
    вдоль траектории от начала до текущей итерации
    """
    return sum(fitness_values[:idx+1])

def _spawn_initial_population(self, n_bacteria: int) -> list[np.ndarray]:
    """
    Инициализация популяции бактерий случайными позициями
    """
    lo, hi = self.bounds[:, 0], self.bounds[:, 1]
    population = []

    for _ in range(n_bacteria):
        candidate = lo + np.random.rand(self.dim) * (hi - lo)
        population.append(candidate.astype(np.float64))

    return population

def run(self) -> Generator[List[np.float64], None, None]:
    """
    Основной цикл алгоритма бактериальной оптимизации

    Процедуры:
    1. Хемотаксис (chemotaxis)
    2. Размножение (reproduction)
    3. Ликвидация и рассеивание (elimination and dispersal)=
    """
    stagnation_cnt = 0
    iter_count = 0
    best_history: list[float] = []

    # Инициализация популяции
    population = [(self._fitness(p), p) for p in
self._spawn_initial_population(self.n_bacteria)]
    population.sort(key=lambda t: t[0])
    yield [x[1] for x in population]

    progress = 0

```



```

while stagnation_cnt < self.stagnation and iter_count < self.max_iter:
    iter_count += 1
    progress = iter_count / self.max_iter

    # Адаптивное уменьшение размера шага
    curr_step_size = self.step_size * (1 - progress) ** 2

    # === ХЕМОТАКСИС (CHEMOTAXIS) ===
    new_population = []
    bacteria_trajectories = {} # Для отслеживания здоровья

    for i, (fitness_val, bacteria_pos) in enumerate(population):
        bacteria_trajectories[i] = [fitness_val]

        for _ in range(self.n_chemotaxis):
            direction = np.random.uniform(-1, 1, size=self.dim)
            candidate = self._chemotaxis_step(bacteria_pos, curr_step_size,
direction)

            candidate_fitness = self._fitness(candidate)

            # Плаваем, пока происходит уменьшение фитнес функции
(улучшение решения)
            if candidate_fitness >= fitness_val:
                bacteria_pos = candidate
                fitness_val = candidate_fitness
            else:
                break

            bacteria_trajectories[i].append(candidate_fitness)

        new_population.append((fitness_val, bacteria_pos))

    # === РАЗМНОЖЕНИЕ (REPRODUCTION) ===
    # Бактерии с хорошим состоянием здоровья делятся
    health_values = []
    for i, (fitness_val, bacteria_pos) in enumerate(new_population):
        health = bacteria_trajectories.get(i, [fitness_val])
        health_values.append(sum(health))

    population = new_population
    population.sort(key=lambda t: t[0], reverse=True)

    # Сортируем по здоровью
    sorted_indices = np.argsort(health_values)[::-1] # По убыванию

    # Делим лучшую половину популяции
    n_reproduce = max(1, self.n_bacteria // self.n_reproduction)
    reproduce_indices = sorted_indices[:n_reproduce]

    reproduced = []
    for idx in reproduce_indices:

```

```

        _, bacteria_pos = population[idx]
        # Новая бактерия наследует позицию родителя с небольшим возмущением
        offspring = bacteria_pos + np.random.uniform(-curr_step_size,
curr_step_size, self.dim)
        offspring = np.clip(offspring, self.bounds[:, 0], self.bounds[:,
1])

        new_bacteria_pos = offspring
        #new_bacteria_pos = bacteria_pos.copy()
        reproduced.append((self._fitness(new_bacteria_pos),
new_bacteria_pos))

        # Объединяем популяцию после размножения (сохраняя размер)
        population = population[:self.n_bacteria - len(reproduced)] +
reproduced

        population.sort(key=lambda t: t[0], reverse=True)

        # === ЛИКВИДАЦИЯ И РАССЕЙВАНИЕ (ELIMINATION AND DISPERSAL) ===
        # С вероятностью случайно выбираем бактерий для ликвидации
        # и рассеивания в случайные точки пространства поиска
        if(self.max_iter - iter_count < 500):
            for _ in range(self.n_elimination):
                if np.random.rand() < self.elim_prob:
                    # Случайная ликвидация
                    idx_eliminate = np.random.randint(0, len(population))
                    new_bacterium = self._random_point()
                    population[idx_eliminate] = (self._fitness(new_bacterium),
new_bacterium)

                    population.sort(key=lambda t: t[0], reverse=True)
                    best_history.append(population[0][0])

        # Выход для визуализации
        dots = [x[1] for x in population][::-1]
        yield dots

        # === ПРОВЕРКА СТАГНАЦИИ ===
        if len(best_history) > self.stagnation_param:
            if abs(best_history[-1] - best_history[-self.stagnation_param]) <
1e-5:
                stagnation_cnt += 1
            else:
                stagnation_cnt = 0

def bacterial_optimization(
    func,
    x_bounds: tuple,
    y_bounds: tuple,
    n_bacteria: int = 20,
    n_chemotaxis: int = 100,
    n_reproduction: int = 4,
    n_elimination: int = 2,

```

```

        step_size:      float = 0.1,
        decay_rate:     float = 0.9,
        max_iter:       int   = 500,
        stagnation:     int   = 20,
        elim_prob:      float = 0.25,
        clear_view=None,
        **func_params
) -> Generator:
    """
    Parameters:
    -----
    func : callable
        Целевая функция для оптимизации
    x_bounds : tuple
        Границы по оси X (min, max)
    y_bounds : tuple
        Границы по оси Y (min, max)
    n_bacteria : int
        Количество бактерий в популяции
    n_chemotaxis : int
        Количество шагов хемотаксиса для каждой бактерии
    n_reproduction : int
        Частота размножения (каждая n-я бактерия размножается)
    n_elimination : int
        Количество процедур ликвидации и рассеивания
    step_size : float
        Начальный размер шага хемотаксиса
    decay_rate : float
        Коэффициент затухания размера шага
    max_iter : int
        Максимальное количество итераций
    stagnation : int
        Порог для проверки стагнации
    elim_prob : float
        Вероятность ликвидации бактерии на каждом шаге
    clear_view : callable, optional
        Функция для очистки визуализации
    **func_params : dict
        Дополнительные параметры целевой функции

    Yields:
    -----
    list
        Список позиций бактерий на текущей итерации
    """
    alg = BacterialOptimization(
        func=func, func_params=func_params, bounds=[x_bounds, y_bounds],
        n_bacteria=n_bacteria, n_chemotaxis=n_chemotaxis,
        n_reproduction=n_reproduction, n_elimination=n_elimination,
        step_size=step_size, decay_rate=decay_rate,
        max_iter=max_iter, stagnation=stagnation, elim_prob=elim_prob,

```

```
)  
for solution in alg.run():  
    if clear_view:  
        clear_view()  
    yield solution
```